default

```
#define silentwitch ios::sync_with_stdio(0),cin.tie(0);
```

ksm

```
  int ksm(int x, int y) {
    int ans = 1;
    while (y) {
        if (y & 1)
            ans = ans * x;
        x = x * x;
        y >>= 1;
    }
    return ans;
}

// 矩陣乘法
vector<vector<int>> mul(const vector<vector<int>> &A,
                        const vector<vector<int>> &B) {
    int n = A.size();
    vector<vector<int>> C(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) {
        for (int k = 0; k < n; k++) if (A[i][k]) {
            for (int j = 0; j < n; j++) {
                C[i][j] = (C[i][j] + A[i][k] * B[k][j]) % MOD;
            }
        }
    }
    return C;
}

// 矩陣快速冪
vector<vector<int>> ksm(vector<vector<int>> base, int exp) {
    int n = base.size();
    vector<vector<int>> res(n, vector<int>(n, 0));

    for (int i = 0; i < n; i++) res[i][i] = 1;

    while (exp > 0) {
        if (exp & 1) res = mul(res, base);
        base = mul(base, base);
        exp >>= 1;
    }
    return res;
}
```

shortest paths

```
// ===== BFS =====
void bfs(int s) {  // 起點
    queue<int> q;  // 這個存走訪名單
    q.push(s);     // s 第一個放入要走訪的名單
    vis[s] = true;

    while (!q.empty()) {
        int u = q.front();   // u 是現在走訪的節點
        q.pop();             // 把 u 從走訪名單上踢掉

        for (int v : g[u]) {          // 把 u 的每個鄰居 v 都放進名單中
            if (vis[v] == false) {    // 如果沒有走訪過這個鄰居
                q.push(v);            // 就把這個鄰居丟進去名單裡面
                vis[v] = true;        // 這樣子 v 就算是走訪過囉
                dis[v] = dis[u] + 1;  // 順便算一下 s 到 v 的距離是多少
            }
        }
    }
}

// ===== DFS 偵測無向圖環 =====
int vis[MXN];
bool dfs(int u, int p) {
    if (vis[u]) return true;  // 遇到祖先就代表有環，回傳 true
    vis[u] = 1;
    for (auto v : g[u]) {
        if (v == p)      // 邊連到父節點: 不理他 (因為這是無向圖)
            continue;
        if (dfs(v, u))
            return true; // 如果在子孫已經出現環，回傳 true
    }
    return false;
}

// ===== DFS 偵測有向圖環 =====
bool vis[MXN] = {0};                // 初始化為 0 (白點)
bool dfs(int u) {
    if (vis[u] == 1) return true;   // 當走到還沒走完的點，就代表有環
    if (vis[u] == 2) return false;  // 走到已經走完的節點，當沒事發生
    vis[u] = 1;                     // 標記這個點還沒被走完 (白點 -> 灰點)
    for (auto v : g[u]) {
        if (dfs(v)) {
            return true;
        }
    }
    vis[u] = 2;                     // 標記這個點已經被走完 (灰點 -> 黑點)
    return false;
}

// ===== Dijkstra =====
void dijkstra(int s, int t) {      // 起點，終點
    int dis[N];
    for(int i = 0; i < N; i++) {   // 初始化
        dis[i] = INF;              // 值要設為比可能的最短路徑權重還要大的值
    }
    dis[s] = 0;
    priority_queue<pii, vector<pii>, greater<pii>> pq; // 以小到大排序
    pq.push({dis[s], s});
    while(!pq.empty()) {
        int u = pq.top().second;
        pq.pop();
        if(vis[u]) continue;
        vis[u] = 1;
        for(auto [v, w] : vec[u]) {
            if(dis[u] + w < dis[v]) {  // 鬆弛
                dis[v] = dis[u] + w;
                pq.push({dis[v], v});
            }
        }
    }
}

// ===== Bellman-Ford =====
void bellman_ford(int n, int m) {
    for(int t = 1; t < n; t++) {   // 注意: 這邊只需要跑 n-1 次，而不是 n 次
        for (int i = 0; i < m; i++) {
            if (dis[edge[i].u] + edge[i].w < dis[edge[i].v]) {
                dis[edge[i].v] = dis[edge[i].u] + edge[i].w;
            }
        }
    }
}

// ===== SPFA =====
struct edge {
    int v, w;
};
int s, n;  // 起點 點數量
vector<edge> E[MXN];
vector<int> dis(MXN, INF), cnt(MXN), inq(MXN);
// cnt記錄最短路經過幾條邊
// inq記錄節點是否在queue裡面
queue<int> Q;

void spfa() {
    dis[s] = 0;
    Q.push(s);
    inq[s] = 1;
    while (Q.size()) {
        int u = Q.front();
        Q.pop();
        inq[u] = 0;
        for (auto [v, w] : E[u]) {
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                cnt[v] = cnt[u] + 1;
                if (cnt[v] >= n) {
                    // 有負環 因為一般最短路只會經過 n - 1 條邊
                }
                if (!inq[v]) Q.push(v), inq[v] = 1;
            }
        }
    }
}

// ===== Floyd-Warshall =====
int dis[N + 1][N + 1];

void floyd_warshall() {
    for(int i = 1; i <= N; i++) {
        for(int j = 1; j <= N; j++) {
            dis[i][j] = INF;  // INF 就是很大的數
        }
        dis[i][i] = 0;
    }
    // 輸入邊的資訊
    int u, v, w;
    cin >> u >> v >> w;
    dis[u][v] = min(dis[u][v], w);

    // Floyd-Warshall 演算法
    for(int k = 1; k <= N; k++) {  // 窮舉中繼點k
        for(int i = 1; i <= N; i++) {
            for(int j = 1; j <= N; j++) {  // 窮舉點對(i,j)
                dis[i][j] = min(dis[i][j], dis[i][k] + dis[k][j]);
            }
        }
    }
}
```

topological sort

```
// ===== 拓撲排序 - 基礎版本 =====
vector<int> edge[MAXN], ans;
int deg[MAXN];

void topo(int n) {
    queue<int> q;

    for (int i = 1; i <= n; i++)
        if (!deg[i])
            q.push(i);

    while (!q.empty()) {
        int u = q.front();
        q.pop();
        ans.push_back(u);

        for (int v : edge[u]) {
            deg[v]--;
            if (!deg[v])
                q.push(v);
        }
    }
}

// ===== 拓撲排序 - 完整版本 with 檢查 =====
void solve(){
    int n, m;
    cin >> n >> m;
    vector<int> edge[m + 5];
    vector<int> deg(n + 5, 0);
    vector<int> ans;

    for(int i = 1; i <= m; i++) {
        int u, v;
        cin >> u >> v;    // 點 u -> v
        edge[u].push_back(v);
        deg[v]++;         // v 的入度 +1
    }

    queue<int> que;
    for(int i = 1; i <= n; i++) {
        if(deg[i] == 0) que.push(i); // 度數為 0 時推入 queue 中
    }

    while (!que.empty()) {
        int u = que.front();
        que.pop();
        ans.push_back(u); // 結果存 ans

        for (int i : edge[u]) {
            deg[i]--;     // 每次走訪完都先幫鄰居的入度 -1
            if (!deg[i])  // 如果入度 == 0 則加入此節點
                que.push(i);
        }
    }

    if(ans.size() != n) {
        cout << "IMPOSSIBLE";  // 有環，無法進行拓撲排序
    }
    else {
        for(int i = 0; i < n; i++) {
            cout << ans[i] << " \n"[i == n];
        }
    }
}

// ===== 拓撲排序 + DP (計數路徑) =====
void solve_dp(){
    int n, m;
```

```cpp
    cin >> n >> m;

    vector<int> edge[n + 5];
    vector<int> deg(n + 5, 0);
    vector<long long> dp(n + 5, 0);

    for(int i = 0; i < m; i++) {
        int u, v;
        cin >> u >> v;
        edge[u].push_back(v);
        deg[v]++;
    }

    dp[1] = 1;  // 初始起點

    queue<int> q;
    for (int i = 1; i <= n; i++) {
        if (deg[i] == 0) {
            q.push(i);
        }
    }

    while (!q.empty()) {
        int u = q.front();
        q.pop();

        for (int i : edge[u]) {
            dp[u] %= (int)1e9 + 7;
            dp[i] = dp[i] + dp[u];  // DP 轉移: 累加所有入邊的路徑數
            dp[i] %= (int)1e9 + 7;
            deg[i]--;

            if (deg[i] == 0)
                q.push(i);
        }
    }

    dp[n] %= (int)1e9 + 7;
    cout << dp[n];  // 輸出到達終點的路徑數
}
```

BIT

```cpp
struct Binary_Indexed_Tree{
    int n;
    vector<long long> bit;
    int lowbit(int x){
        return x&-x;
    }
    void init(int _n){
        n = _n+1;
        bit = vector<long long>(n,0);
    }
    void update(int x,int v){
        for(; x<n; x+=lowbit(x)){
            bit[x] += v;
        }
    }
    long long query(int x){
        long long ret = 0;
        for(; x>0; x-=lowbit(x)){
            ret += bit[x];
        }
        return ret;
    }
}BIT;

// ===== 離散化 =====
// vector<int> arr;
vector<int> tmp(arr);
sort(tmp.begin(), tmp.end());
tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
for (int i = 0; i < n; i++)
    arr[i] = lower_bound(tmp.begin(), tmp.end(), arr[i]) - tmp.begin();

// ===== Implementation 1: 逆序对计数 =====
// 统计数组中的逆序对数量
// 逆序对: 对于 i < j，如果 a[i] > a[j]，则 (i,j) 为一个逆序对
void solve1(){
    int n;
    cin >>n;
    int ans = 0;
    BIT.init(1000005);
    // 遍历数组
    for(int i = 1 ; i <= n; i ++ ){
        int a;
        cin >> a;
        if(a==0){
            continue;
        }
        else {
            // 查询有多少个已经出现过且比 a 小的数
            ans += BIT.query(a-1);
            // 将当前数字 a 加入到 BIT 中
            BIT.update(a,a);
        }
    }
    cout << ans << endl;
}

// ===== Implementation 2: 区间更新与单点查询 =====
// 支持对一个区间内的所有元素加上同一个值
// 以及查询单个位置的值
void solve2(){
    int n,q;
    cin >> n >> q;
    int pre = 0;
    int a;
    BIT.init(n);
    // 读取初始数组，存储差分值
    for(int i = 1;i <= n; i++){
        cin >> a;
        BIT.update(i,a-pre);
        pre = a;
    }
    // 处理 q 个查询
    for(int i = 0 ;i<q;i++){
        int t;
        cin >> t;
        if(t == 1){
            // 区间更新: 对 [a, b] 区间加上 u
            int b,u;
            cin >> a >> b >> u;
            BIT.update(a,u);
            BIT.update(b+1,-u);
        }
        else{
            // 单点查询: 查询位置 a 的值
            cin >> a;
            cout << BIT.query(a)<<endl;
        }
    }
}

// ===== Implementation 3: 离散化 + 平衡查询 =====
// 用两个数组 a[i] 和 b[i] 表示某种属性
// 离散化后，检查是否满足某个平衡条件
void solve3(){
    int n,k;
    cin >> n>> k;
    vector<pair<int , int>> prefect(n+1);
    vector<int> a(n+1),b(n+1),tmp(n+1,0);
    for(int i = 1 ;i<=n;i++){
        cin >> a[i] >> b[i];
    }

    // 对 a[] 进行离散化处理
    for(int i = 1; i <= n; i++) tmp[i] = a[i];
    sort(tmp.begin() + 1, tmp.begin() + n + 1);
    int len = unique(tmp.begin() + 1, tmp.begin() + n + 1) - (tmp.begin() + 1);
    for(int i = 1; i <= n; i++)
        a[i] = lower_bound(tmp.begin() + 1, tmp.begin() + 1 + len, a[i]) - (tmp.begin() +
                1) + 1;

    // 对 b[] 进行离散化处理
    for(int i = 1; i <= n; i++) tmp[i] = b[i];
    sort(tmp.begin() + 1, tmp.begin() + n + 1);
    int lenb = unique(tmp.begin() + 1, tmp.begin() + n + 1) - (tmp.begin() + 1);
    for(int i = 1; i <= n; i++)
        b[i] = lower_bound(tmp.begin() + 1, tmp.begin() + 1 + lenb, b[i]) - (tmp.begin()
                + 1) + 1;

    // 组合成对并排序
    for(int i  = 1;i<=n;i++){
        prefect[i] = {a[i],b[i]};
    }
    sort(all(prefect));

    vector<int> corn(n+1);
    BIT.init(n);
    int num = 0;
    int conr;
    for(int i = 1 ;i <= n;i++){
        // 统计前面有多少个 b 值小于当前 b[i]
        conr = BIT.query(prefect[i].second-1);
        BIT.update(prefect[i].second,1);

        // 计算平衡度: 左边比当前小的数 vs 右边比当前小的数
        int gate = abs(conr -(n-1-conr));
        if(gate >= k){
            num++;
        }
    }
    cout << num << endl;
}

// ===== Implementation 4: 高级逆序对计数 =====
// 计算对于每个元素，有多少个数对 (i,j)，其中 i < j
// 且 a[i], a[j] 相对于当前元素形成某种关系
void solve4(){
    int n;
    cin >> n;
    vector<int> a(n+1,0);
    for(int i = 1;i<=n;i++){
        cin >> a[i];
    }

    // 离散化: 将原数组值映射到 1~len 的范围
    vector<int> tmp(a.begin()+1,a.end());
    sort(tmp.begin(), tmp.end());
    tmp.erase(unique(tmp.begin(), tmp.end()), tmp.end());
    for (int i = 1; i <= n; i++)
        a[i] = (lower_bound(tmp.begin(), tmp.end(), a[i]) - tmp.begin())+1;

    // 计算 left[i]: 在位置 i 左边有多少个数比 a[i] 大
    vector<int> left(n+1,0),right(n+1,0);
    BIT.init(n);
    for(int i =1 ;i<=n;i++){
        left[i] = (i-1) - BIT.query(a[i]);
        BIT.update(a[i],1);
    }

    // 计算 right[i]: 在位置 i 右边有多少个数比 a[i] 小
    BIT.init(n);
    for(int i = n;i>0;i--){
        right[i] = (a[i]>1)?BIT.query(a[i]-1):0;
        BIT.update(a[i],1);
    }

    // 答案 = sum(left[i] * right[i])
    // 表示有多少个三元组 (i,j,k) 满足 i<j<k，且 a[i]>a[j]<a[k]
    int ans = 0;
    for(int i = 1;i<=n;i++){
        ans += right[i]*left[i];
    }
    cout << ans;
}
```

Big Cheat

```cpp
// Codeforce 1 - KSM + 位操作
const int mod = 1e9+7;
int ksm(int b,int e){
    int res = 1; b%=mod;
    while(e>0){
        if(e%2==1) res = (res*b)%mod;
        b = (b*b)%mod; e/=2;
    }
    return res;
}
void solve_cf1(){
    int n,k; cin >> n >> k;
    vector<int> a(n);
    vector<vector<int>> one(n);
    int mxbit = 0;
    for(int i = 0; i < n; i++){
        cin >> a[i];
        for(int b = 0; (a[i] >> b) > 0; b++)
            if((a[i] >> b) & 1){
                one[i].push_back(b);
                mxbit = max(mxbit, b);
            }
    }
    int ans = 0;
    for(int j = mxbit;j>=0;j--){
        int cost = 0;
        for(int x = 0 ; x < n ; x++){
            auto it = upper_bound(one[x].begin(),one[x].end(),j);
            if(it == one[x].begin()){ cost = LLONG_MAX; break; }
            it--; cost += (j-*it);
            if(cost > k) break;
        }
```

```
        if(cost <=k){
            for(int x = 0 ; x < n ; x++){
                auto it = upper_bound(one[x].begin(),one[x].end(),j);
                it--; one[x].erase(it);
            }
            k-=cost; int leftshift = k/n;
            ans += ksm(2,j+leftshift);
            k-=n*leftshift;
        }
    }
    cout << ans% ((int)1e9 + 7);
}
// Codeforce 2 - DP 騎士路徑計數
int dp[1000005][4][4] = {0};
bool check(int x,int y){
    return x>=0 && y >=0 && x<1000000 && y < 4;
}
void solve_cf2(){
    const int mod = 1000000007;
    int n; cin >> n;
    int dx[4] = { 2 , 2 , 1 , 1 };
    int dy[4] = {-1 , 1 ,-2 , 2 };
    memset(dp, 0, sizeof(dp));
    for(int i = 0;i<4;i++){
        int nx = 0 + dx[i];
        int ny = 0 + dy[i];
        if(check(nx,ny)) dp[nx][ny][i] = 1;
    }
    for(int i = 1; i < n; i++) {
        for(int j = 0; j < 4; j++) {
            for(int lk = 0; lk < 4; lk++) {
                if(dp[i][j][lk] == 0) continue;
                for(int k = 0; k < 4; k++) {
                    if(k == lk) continue;
                    int nx = i + dx[k];
                    int ny = j + dy[k];
                    if(check(nx, ny))
                        dp[nx][ny][k] = (dp[nx][ny][k] + dp[i][j][lk]) % mod;
                }
            }
        }
    }
    int ans = 0;
    for(int i = 0;i<4;i++) ans = (ans + dp[n-1][3][i])%mod;
    cout << ans << endl;
}
```

Short path question

```
#define int long long
#define INF 1e18
#define pii pair<int,int>

struct Edge {
    int u, v, w;
};

// ===== Implementation 1: Dijkstra - 求最短路径距离 =====
// 从源点到所有点的最短路径
void solve1(){
    int n,m;
    cin >> n >> m;
    vector<int> dis(n+5,0),vis(n+5,0);
    vector<vector<pii>>  vec(n+5); // pair 存 {終點,權重}
    for(int i = 0 ;i<=n;i++){
        dis[i] = INF;
    }
    dis[1] = 0;

    // 读取边的信息
    for(int i = 0 ;i<m;i++){
        int a,b,c;
        cin >> a >> b >> c;
        vec[a].push_back(make_pair(b,c));
    }

    // 使用优先队列实现 Dijkstra，按距离从小到大排序
    priority_queue<pii,vector<pii>,greater<pii>> pq;
    pq.push({dis[1],1});

    while(pq.empty() == 0){
        int u = pq.top().second;
        pq.pop();

        // 若已访问过则跳过
        if(vis[u])continue;
        vis[u] = 1;

        // 对所有相邻节点进行松弛操作
        for(int i = 0 ;i  <vec[u].size();i++){
            int v = vec[u][i].first;
            int w = vec[u][i].second;
            if(dis[u]+w<dis[v]){
                dis[v]=dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }

    // 输出所有点的最短距离
    for(int i = 1 ;i<=n;i++){
        cout << dis[i]<<" ";
    }
}

// ===== Implementation 2: Dijkstra - 记录最短路径 =====
// 记录每个点的前驱边编号，用于重构最短路径
void solve2(){
    int n,m;
    cin >> n >> m;

    vector<int> dis(n+5), where_he_from(n+5), vis(n+5,0);

    // 存储 {终点, {权重, 边编号}}
    vector<vector<pair<int,pair<int,int>>>> vec(n+5);

    for(int i=1;i<=n;i++) dis[i]=INF;
    dis[1]=0;

    // 读取无向边
    for(int i=1;i<=m;i++){
        int a,b,c;
        cin >> a >> b >> c;
        vec[a].push_back({b,{c,i}});
        vec[b].push_back({a,{c,i}});
    }

    priority_queue<pii,vector<pii>,greater<pii>> pq;
    pq.push({0,1});

    while(!pq.empty()){
        int u = pq.top().second;
        pq.pop();

        if(vis[u]) continue;
        vis[u] = 1;

        // 对相邻节点进行松弛，并记录前驱边
        for(int i = 0 ;i<vec[u].size();i++){
            int v = vec[u][i].first;
            int w = vec[u][i].second.first;
            int edge_id = vec[u][i].second.second;

            if(dis[u]+w<dis[v]){
                where_he_from[v] = edge_id;
                dis[v]=dis[u]+w;
                pq.push({dis[v],v});
            }
        }
    }

    // 输出每个节点（除了源点）的前驱边编号
    for(int i=2;i<=n;i++){
        cout << where_he_from[i] << " ";
    }
}

// ===== Implementation 3: Floyd-Warshall 算法 =====
// 求所有点对之间的最短路径
void solve3(){
    int n,m,q;
    cin >> n >> m >> q;
    int N = n;
    int dis[N + 1][N + 1];

    // 初始化：同一点距离为 0，其他为无穷大
    for(int i = 1; i <= N; i++){
        for(int j = 1; j <= N; j++){
            dis[i][j] = INF;
        }
        dis[i][i] = 0;
    }

    // 读取边信息（双向边，取较小值）
    while(m--){
        int a, b, c;
        cin >> a >> b >> c;
        dis[a][b]=min(dis[a][b], c);
        dis[b][a]=min(dis[b][a], c);
    }

    // Floyd-Warshall 主算法：枚举中间点 k
    for(int k=1;k<=N;k++){
        // 枚举所有点对 (i,j)
        for(int i=1;i<=N;i++){
            for(int j=1;j<=N;j++){
                // 尝试通过点 k 的路径
                dis[i][j]=min(dis[i][j],dis[i][k]+dis[k][j]);
            }
        }
    }

    // 处理 q 个查询
    while(q--){
        int a,b;
        cin >> a >> b;
        if(dis[a][b]!=INF){
            cout << dis[a][b]<<endl;
        }
        else cout << -1 << endl;
    }
}

// ===== Implementation 4: Bellman-Ford 算法 =====
// 可以处理负权边，并检测负环
void solve4(){
    const long long INF2 = (1LL<<60);
    int n,m;
    cin >> n >> m;

    struct Edge {
        int u, v, w;
    };
    vector<Edge> e(m);
    vector<int> dis(n + 1, INF2),neg(n+1,0);

    // 读取边，权重取负（用于处理最大值问题）
    for (int i = 0; i < m; i++){
        cin >> e[i].u >> e[i].v >> e[i].w;
        e[i].w = -e[i].w;
    }
    dis[1]=0;

    // 第一阶段：放松所有边 n-1 次
    for (int t = 1; t < n; t++) {
        for (int i = 0; i < m; i++) {
            int u = e[i].u, v = e[i].v, w = e[i].w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
            }
        }
    }

    // 第二阶段：再松弛一次，检测负环
    // 如果还能松弛则存在负环，标记受影响的节点
    for (int t = 1; t < n; t++) {
        for (int i = 0; i < m; i++) {
            int u = e[i].u, v = e[i].v, w = e[i].w;
            if (dis[v] > dis[u] + w) {
                dis[v] = dis[u] + w;
                neg[v] = 1;
            }
            // 从负环影响的点出发的边也受负环影响
            if (neg[u]) neg[v] = 1;
        }
    }

    // 如果终点受到负环影响，输出 inf；否则输出最短路径
    if(neg[n]){
        cout <<"inf";
    }
    else cout << -dis[n];
}
```

graph

```
// Topological Sort - 基礎版本
void solve(){
int n, m; cin >> n >> m;
vector<int> edge[m + 5];
vector<int> deg(n + 5, 0);
vector<int> ans;
```

```
for(int i = 1; i <= m; i++) {
  int u, v; cin >> u >> v;
  edge[u].push_back(v);
  deg[v]++;
}
queue<int> que;
for(int i = 1; i <= n; i++)
  if(deg[i] == 0) que.push(i);
while (!que.empty()) {
  int u = que.front(); que.pop();
  ans.push_back(u);
  for (int i : edge[u]) {
    deg[i]--;
    if (!deg[i]) que.push(i);
  }
}
if(ans.size() != n) cout << "IMPOSSIBLE";
else for(int i = 0; i < n; i++) cout << ans[i] << " \n"[i == n];
}
// Topological Sort - DP版本 (計數路徑)
void solve_dp(){
int n, m; cin >> n >> m;
vector<int> edge[n + 5];
vector<int> deg(n + 5, 0);
vector<long long> dp(n + 5, 0);
for(int i = 0; i < m; i++) {
  int u, v; cin >> u >> v;
  edge[u].push_back(v);
  deg[v]++;
}
dp[1] = 1;
queue<int> q;
for (int i = 1; i <= n; i++)
  if (deg[i] == 0) q.push(i);
while (!q.empty()) {
  int u = q.front(); q.pop();
  for (int i : edge[u]) {
    dp[u] %= (int)1e9 + 7;
    dp[i] = dp[i] + dp[u];
    dp[i] %= (int)1e9 + 7;
    deg[i]--;
    if (deg[i] == 0) q.push(i);
  }
}
dp[n] %= (int)1e9 + 7;
cout << dp[n];
}
```

codeforce

```
// Codeforce 1 - KSM + 位操作
const int mod = 1e9+7;
int ksm(int b,int e){
    int res = 1; b%=mod;
    while(e>0){
        if(e%2==1) res = (res*b)%mod;
        b = (b*b)%mod; e/=2;
    }
    return res;
}
void solve_cf1(){
    int n,k; cin >> n >> k;
    vector<int> a(n);
    vector<vector<int>> one(n);
    int mxbit = 0;
    for(int i = 0; i < n; i++){
        cin >> a[i];
        for(int b = 0; (a[i] >> b) > 0; b++)
            if((a[i] >> b) & 1){
                one[i].push_back(b);
                mxbit = max(mxbit, b);
            }
    }
    int ans = 0;
    for(int j = mxbit;j>=0;j--){
        int cost = 0;
        for(int x = 0 ; x < n ; x++){
            auto it = upper_bound(one[x].begin(),one[x].end(),j);
            if(it == one[x].begin()){ cost = LLONG_MAX; break; }
            it--; cost += (j-*it);
            if(cost > k) break;
        }
        if(cost <=k){
            for(int x = 0 ; x < n ; x++){
                auto it = upper_bound(one[x].begin(),one[x].end(),j);
                it--; one[x].erase(it);
            }
            k-=cost; int leftshift = k/n;
            ans += ksm(2,j+leftshift);
            k-=n*leftshift;
        }
    }
    cout << ans% ((int)1e9 + 7);
}
// Codeforce 2 - DP 騎士路徑計數
int dp[1000005][4][4] = {0};
bool check(int x,int y){
    return x>=0 && y >=0 && x<1000000 && y < 4;
}
void solve_cf2(){
    const int mod = 1000000007;
    int n; cin >> n;
    int dx[4] = { 2 , 2 , 1 , 1 };
    int dy[4] = {-1 , 1 ,-2 , 2 };
    memset(dp, 0, sizeof(dp));
    for(int i = 0;i<4;i++){
        int nx = 0 + dx[i];
        int ny = 0 + dy[i];
        if(check(nx,ny)) dp[nx][ny][i] = 1;
    }
    for(int i = 1; i < n; i++) {
        for(int j = 0; j < 4; j++) {
            for(int lk = 0; lk < 4; lk++) {
                if(dp[i][j][lk] == 0) continue;
                for(int k = 0; k < 4; k++) {
                    if(k == lk) continue;
                    int nx = i + dx[k];
                    int ny = j + dy[k];
                    if(check(nx, ny))
                        dp[nx][ny][k] = (dp[nx][ny][k] + dp[i][j][lk]) % mod;
                }
            }
        }
    }
    int ans = 0;
    for(int i = 0;i<4;i++) ans = (ans + dp[n-1][3][i])%mod;
    cout << ans << endl;
}
```